

tuvis.py API Reference

Plotly-based Visualization Toolkit for Game Mathematics

Version 1.0 — April 15, 2026

동명대학교 게임학부 강영민

Source: <https://github.com/dknife/linalg/blob/main/tool/tuvis.py>

Contents

1	Overview	2
1.1	Installation	2
1.2	Import	2
1.3	Google Colab에서 사용하기	2
1.4	Common Parameters	2
2	Figure Creation	2
3	Vectors	4
4	Points	5
5	Matrix Visualization	6
6	Shapes	8
7	Lines	9
8	Planes	10
9	Bounding Objects	12
10	Quick Reference	13

1 Overview

tuvis.py는 **plotly** 기반의 2D/3D 시각화 도구 모듈이다. WebGL Z-buffer를 사용하므로 matplotlib과 달리 깊이(depth)가 정확하게 처리되며, 브라우저에서 인터랙티브하게 회전/줌/팬이 가능하다. Google Colab에서도 동작한다.

1.1 Installation

```
pip install plotly numpy
```

1.2 Import

```
from tuvis import *  
# or  
import tuvis
```

1.3 Google Colab에서 사용하기

Colab 환경에서는 첫 번째 셀에서 아래 코드를 실행하면 바로 사용할 수 있다.

```
!wget -q 'https://raw.githubusercontent.com/dknife/linalg/main/tool/tuvis.py'  
from tuvis import *  
!rm tuvis.py
```

wget으로 GitHub에서 최신 tuvis.py를 다운로드하고, import 한 뒤 파일을 삭제하여 작업 디렉터리를 깔끔하게 유지한다. 이후 셀에서 tuvis의 모든 함수를 바로 사용할 수 있다.

1.4 Common Parameters

모든 선 그리기 함수에서 공통으로 사용되는 파라미터:

- **color** (str, default: 'red') — 색상 문자열 (예: 'red', 'blue', '#FF5500')
- **alpha** (float, default: 1.0) — 투명도 (0 = 완전 투명, 1 = 불투명)
- **lineType** (str, default: 'solid') — 선 스타일: 'solid', 'dotted', 'dashed'

2 Figure Creation

figure2d

2D 가시화를 위한 Figure를 생성한다. xy 평면 위에 배경, 그리드, 축선을 그리고 정사영(orthographic) 카메라를 설정한다.

Signature:

```
fig = figure2d(x=[-1,1], y=[-1,1], title='',  
               width=600, height=600,  
               bg_color='rgba(248, 250, 255, 0.3)')
```

Parameters:

- **x** (list, default: [-1, 1]) — x축 범위 [xmin, xmax]
- **y** (list, default: [-1, 1]) — y축 범위 [ymin, ymax]
- **title** (str, default: '') — Figure 제목
- **width** (int, default: 600) — Figure 너비 (pixels)
- **height** (int, default: 600) — Figure 높이 (pixels)
- **bg_color** (str, default: 'rgba(248,250,255,0.3)') — 배경 색상

Returns: go.Figure

Example:

```
fig = figure2d(x=[-5, 5], y=[-5, 5], title='2D Canvas')
draw_vec2d(fig, [3, 2], color='red', label='v')
fig.show()
```

figure3d

3D 가시화를 위한 Figure를 생성한다. 등비율(cube) aspect로 설정된다.

Signature:

```
fig = figure3d(x=[-1,1], y=[-1,1], z=[-1,1],
               title='', width=600, height=600)
```

Parameters:

- **x** (list, default: [-1, 1]) — x축 범위
- **y** (list, default: [-1, 1]) — y축 범위
- **z** (list, default: [-1, 1]) — z축 범위
- **title** (str, default: '') — Figure 제목
- **width** (int, default: 600) — Figure 너비 (pixels)
- **height** (int, default: 600) — Figure 높이 (pixels)

Returns: go.Figure

Example:

```
fig = figure3d(x=[-3,3], y=[-3,3], z=[-3,3], title='3D Space')
draw_vec3d(fig, [1, 2, 3], color='blue')
fig.show()
```

setCam

카메라를 LookAt 방식으로 설정한다.

Signature:

```
setCam(fig, eye, target=(0,0,0), up=(0,0,1))
```

Parameters:

- **fig** (go.Figure, **required**) — 대상 Figure
- **eye** (tuple/list, **required**) — 카메라 위치 (x, y, z)
- **target** (tuple, default: $(0, 0, 0)$) — 바라볼 지점
- **up** (tuple, default: $(0, 0, 1)$) — 월드 업 벡터

Example:

```
setCam(fig, eye=(2, -2, 1.5), target=(0, 0, 0), up=(0, 0, 1))
```

3 Vectors

draw_vec2d

2D 벡터를 $z = 0$ 평면 위에 화살표로 그린다.

Signature:

```
draw_vec2d(fig, v, color='red', start_from=None,
            alpha=1.0, label=None,
            cone_ratio=0.12, cone_radius=0.04,
            lineType='solid')
```

Parameters:

- **fig** (go.Figure, **required**) — 대상 Figure
- **v** (array-like, **required**) — 2D 벡터 $[v_x, v_y]$
- **color** (str, default: 'red') — 색상
- **start_from** (array-like, default: None) — 시작점 (기본: 원점)
- **alpha** (float, default: 1.0) — 투명도
- **label** (str, default: None) — 중점에 표시할 텍스트
- **cone_ratio** (float, default: 0.12) — 화살촉 길이 비율
- **cone_radius** (float, default: 0.04) — 화살촉 반지름
- **lineType** (str, default: 'solid') — 선 스타일

Example:

```
fig = figure2d(x=[-1, 5], y=[-1, 5])
a = [3, 1]
b = [1, 3]
draw_vec2d(fig, a, color='red', label='a')
draw_vec2d(fig, b, color='blue', label='b')
draw_vec2d(fig, b, start_from=a, color='blue', alpha=0.4)
fig.show()
```

draw_vec3d

3D 벡터를 화살표(shaft + cone)로 그린다.

Signature:

```
draw_vec3d(fig, v, color='red', start_from=None,
            alpha=1.0, label=None,
            cone_ratio=0.12, cone_radius=0.06,
            lineType='solid')
```

Parameters:

- **fig** (go.Figure, required) — 대상 Figure
- **v** (array-like, required) — 3D 벡터 $[v_x, v_y, v_z]$
- **color** (str, default: 'red') — 색상
- **start_from** (array-like, default: None) — 시작점 (기본: 원점)
- **alpha** (float, default: 1.0) — 투명도
- **label** (str, default: None) — 중점에 표시할 텍스트
- **cone_ratio** (float, default: 0.12) — 벡터 길이 대비 화살촉 비율
- **cone_radius** (float, default: 0.06) — 화살촉 밑면 반지름
- **lineType** (str, default: 'solid') — 선 스타일

Example:

```
fig = figure3d(x=[-3,3], y=[-3,3], z=[-3,3])
u = [2, 0, 0]
v = [0, 2, 1]
w = np.cross(u, v)
draw_vec3d(fig, u, color='red', label='u')
draw_vec3d(fig, v, color='green', label='v')
draw_vec3d(fig, w, color='blue', label='u x v', lineType='dashed')
fig.show()
```

4 Points

draw_points

2D 또는 3D 점 리스트를 그린다. 2D 점은 $z = 0$ 에 배치된다.

Signature:

```
draw_points(fig, points_list, labels=None,
            color='red', size=2)
```

Parameters:

- **fig** (go.Figure, required) — 대상 Figure
- **points_list** (list, required) — 점 리스트 (각 원소는 2D 또는 3D array)
- **labels** (list, default: None) — 각 점에 표시할 라벨 문자열 리스트
- **color** (str, default: 'red') — 점 색상
- **size** (int, default: 2) — 점 크기

Example:

```
pts = [[1, 2], [3, 4], [2, 1]]
draw_points(fig, pts, labels=['P1', 'P2', 'P3'], color='darkred')
```

draw_points_in_matrix

행렬의 열(column)을 점으로 그린다. $2 \times N$ 또는 $3 \times N$ 행렬을 지원한다.

Signature:

```
draw_points_in_matrix(fig, M, color='red', size=2)
```

Parameters:

- **fig** (go.Figure, required) — 대상 Figure
- **M** (np.ndarray, required) — $2 \times N$ 또는 $3 \times N$ 행렬
- **color** (str, default: 'red') — 점 색상
- **size** (int, default: 2) — 점 크기

Example:

```
M = np.array([[0, 1, 2, 3],
               [0, 1, 0, 1]])
draw_points_in_matrix(fig, M, color='blue')
```

5 Matrix Visualization

draw_mat22

2×2 행렬을 $z = 0$ 평면 위에 평행사변형으로 시각화한다. 두 열벡터를 화살표로, 나머지 두 변을 회색 보조선으로 그린다.

Signature:

```
draw_mat22(fig, M, label=None)
```

Parameters:

- **fig** (go.Figure, required) — 대상 Figure
- **M** (np.ndarray, required) — 2×2 행렬
- **label** (str, default: None) — $(u + v)$ 위치에 표시할 텍스트

Example:

```
M = np.array([[2, -1],
              [1,  2]])
draw_mat22(fig, M, label='M')
```

draw_mat33

3×3 행렬을 평행육면체(parallelepiped)로 시각화한다. 세 열벡터(빨강/초록/파랑) + 9개의 보조 모서리(회색)를 그린다.

Signature:

```
draw_mat33(fig, M, label=None)
```

Parameters:

- **fig** (go.Figure, required) — 대상 Figure
- **M** (np.ndarray, required) — 3×3 행렬
- **label** (str, default: None) — $(u + v + w)$ 위치에 표시할 텍스트

Example:

```
M = np.array([[2, 0, 0.5],
              [0, 1, 0.5],
              [0, 0, 2  ]])
draw_mat33(fig, M, label='M')
```

draw_space_mat22

2×2 행렬이 만드는 변환된 격자 공간을 그린다. draw_mat22를 호출한 뒤, 격자선을 추가로 그린다.

Signature:

```
draw_space_mat22(fig, M, label=None,
                 color='gray', alpha=0.2)
```

Parameters:

- **fig** (go.Figure, required) — 대상 Figure
- **M** (np.ndarray, required) — 2×2 행렬
- **label** (str, default: None) — 라벨
- **color** (str, default: 'gray') — 격자선 색상
- **alpha** (float, default: 0.2) — 격자선 투명도

Example:

```
theta = np.radians(30)
R = np.array([[np.cos(theta), -np.sin(theta)],
              [np.sin(theta),  np.cos(theta)]])
draw_space_mat22(fig, R, label='R(30)', color='steelblue')
```

6 Shapes

draw_polygons

다각형 리스트를 그린다. 각 다각형은 $N \times 3$ 배열이며 fan 방식으로 삼각화된다.

Signature:

```
draw_polygons(fig, polygon_list, facecolors, alpha=0.8)
```

Parameters:

- **fig** (go.Figure, required) — 대상 Figure
- **polygon_list** (list, required) — 다각형 리스트 (각 원소: $N \times 3$ array)
- **facecolors** (list, required) — 각 다각형의 면 색상 리스트
- **alpha** (float, default: 0.8) — 투명도

Example:

```
tris = [  
    np.array([[0,0,0], [2,0,0], [1,2,0]]),  
    np.array([[0,0,0], [1,2,0], [0,0,2]]),  
]  
draw_polygons(fig, tris, facecolors=['cyan', 'magenta'])
```

draw_circle_2d

$z = 0$ 평면 위에 원을 그린다 (채움 + 테두리).

Signature:

```
draw_circle_2d(fig, center=(0,0), radius=1.0,  
               color='blue', alpha=0.5,  
               n_segments=64, lineType='solid')
```

Parameters:

- **fig** (go.Figure, required) — 대상 Figure
- **center** (tuple, default: (0,0)) — 중심 좌표
- **radius** (float, default: 1.0) — 반지름
- **color** (str, default: 'blue') — 색상
- **alpha** (float, default: 0.5) — 투명도
- **n_segments** (int, default: 64) — 원 분할 수
- **lineType** (str, default: 'solid') — 테두리 선 스타일

Example:

```
draw_circle_2d(fig, center=(1, 1), radius=2.0,  
               color='red', lineType='dashed')
```


draw_circle_3d

3D 공간에서 임의의 평면 위에 원을 그린다. 법선 벡터로 평면 방향을 지정한다.

Signature:

```
draw_circle_3d(fig, center, normal, radius,
               color='blue', alpha=0.3,
               n_segments=64, lineType='solid')
```

Parameters:

- **fig** (go.Figure, required) — 대상 Figure
- **center** (array-like, required) — 중심 (x, y, z)
- **normal** (array-like, required) — 법선 벡터 (n_x, n_y, n_z)
- **radius** (float, required) — 반지름
- **color** (str, default: 'blue') — 색상
- **alpha** (float, default: 0.3) — 투명도
- **n_segments** (int, default: 64) — 원 분할 수
- **lineType** (str, default: 'solid') — 테두리 선 스타일

Example:

```
draw_circle_3d(fig, center=[0,0,1], normal=[0,0,1],
               radius=2, color='green', alpha=0.4)
```

7 Lines

draw_line

두 점을 잇는 선분(segment), 반직선(ray), 또는 직선(line)을 그린다. ray는 끝에, line은 양 끝에 화살촉이 붙는다. 2D/3D 입력 모두 지원.

Signature:

```
draw_line(fig, p1, p2, label=None,
          type='segment', color='red', alpha=0.5,
          scale=2, cone_radius=0.05,
          lineType='solid')
```

Parameters:

- **fig** (go.Figure, **required**) — 대상 Figure
- **p1** (array-like, **required**) — 시작점 (2D 또는 3D)
- **p2** (array-like, **required**) — 끝점 (2D 또는 3D)
- **label** (str, default: None) — 중점에 표시할 텍스트
- **type** (str, default: 'segment') — 종류:
 - 'segment' — p_1 에서 p_2 까지의 선분
 - 'ray' — p_1 에서 p_2 방향으로 **scale**배 연장한 반직선
 - 'line' — 중점 기준 양방향으로 **scale**배 연장한 직선
- **color** (str, default: 'red') — 색상
- **alpha** (float, default: 0.5) — 투명도
- **scale** (float, default: 2) — ray/line의 연장 배율
- **cone_radius** (float, default: 0.05) — 화살촉 크기
- **lineType** (str, default: 'solid') — 선 스타일

Example:

```
p1, p2 = [1, 1], [4, 3]

# segment
draw_line(fig, p1, p2, label='seg', type='segment', color='blue')

# ray (p1 -> p2 direction, 3x length)
draw_line(fig, p1, p2, label='ray', type='ray',
          color='green', scale=3)

# line (both directions, 3x length, dashed)
draw_line(fig, p1, p2, label='line', type='line',
          color='red', scale=3, lineType='dashed')
```

8 Planes

draw_plane

세 점이 정의하는 삼각형과, 그 평면 위에서 삼각형 중심(centroid)을 중심으로 한 반지름 r 의 원을 함께 그린다.

Signature:

```
draw_plane(fig, p1, p2, p3, r=1,
          triangle_color='cyan',
          circle_color='yellow',
          triangle_alpha=0.7,
          circle_alpha=0.3,
          lineType='solid')
```

Parameters:

- **fig** (go.Figure, **required**) — 대상 Figure
- **p1, p2, p3** (array-like, **required**) — 삼각형 꼭짓점 (3D)
- **r** (float, default: 1) — 평면 원의 반지름
- **triangle_color** (str, default: 'cyan') — 삼각형 면 색상
- **circle_color** (str, default: 'yellow') — 원 색상
- **triangle_alpha** (float, default: 0.7) — 삼각형 투명도
- **circle_alpha** (float, default: 0.3) — 원 투명도
- **lineType** (str, default: 'solid') — 삼각형 테두리 및 원주 선 스타일

Example:

```
fig = figure3d(x=[-3,5], y=[-3,5], z=[-3,5])
draw_plane(fig,
            p1=[0, 0, 0], p2=[3, 0, 1], p3=[1, 2, 2],
            r=5, lineType='dashed')
fig.show()
```

draw_plane_from_normal

한 점 p 를 지나고 법선 $\mathbf{n}$ 을 가지는 평면을 원으로 그린다. 법선 벡터도 화살표로 표시한다.

Signature:

```
draw_plane_from_normal(fig, p, n, r=3,
                      plane_color='yellow',
                      plane_alpha=0.2,
                      lineType='solid')
```

Parameters:

- **fig** (go.Figure, **required**) — 대상 Figure
- **p** (array-like, **required**) — 평면 위의 점 (x, y, z)
- **n** (array-like, **required**) — 법선 벡터 (n_x, n_y, n_z)
- **r** (float, default: 3) — 원 반지름
- **plane_color** (str, default: 'yellow') — 원 색상
- **plane_alpha** (float, default: 0.2) — 원 투명도
- **lineType** (str, default: 'solid') — 원주 및 법선 벡터 선 스타일

Example:

```
draw_plane_from_normal(fig, p=[1,1,1], n=[0,1,0],
                      r=3, lineType='dotted')
```

9 Bounding Objects

draw_bounding_sphere

바운딩 구(Bounding Sphere)를 그린다.

Signature:

```
draw_bounding_sphere(fig, center, radius,
                     color='blue', alpha=0.3)
```

Parameters:

- **fig** (go.Figure, required) — 대상 Figure
- **center** (tuple/list, required) — 중심 (x, y, z)
- **radius** (float, required) — 반지름
- **color** (str, default: 'blue') — 표면 색상
- **alpha** (float, default: 0.3) — 투명도

Example:

```
draw_bounding_sphere(fig, center=(0, 0, 0), radius=2,
                     color='blue', alpha=0.2)
```

draw_aabb

축 정렬 바운딩 박스(AABB)를 그린다.

Signature:

```
draw_aabb(fig, min_pt, max_pt,
          color='green', alpha=0.2)
```

Parameters:

- **fig** (go.Figure, required) — 대상 Figure
- **min_pt** (tuple/list, required) — 최소점 ($x_{\min}, y_{\min}, z_{\min}$)
- **max_pt** (tuple/list, required) — 최대점 ($x_{\max}, y_{\max}, z_{\max}$)
- **color** (str, default: 'green') — 면 색상
- **alpha** (float, default: 0.2) — 투명도

Example:

```
draw_aabb(fig, min_pt=(-1, -1, -1), max_pt=(2, 2, 2),
          color='green', alpha=0.15)
```

draw_obb

방향 바운딩 박스(OBB)를 그린다. 로컬 축 3개도 함께 표시된다.

Signature:

```
draw_obb(fig, center, half_extents, rotation,
```

```
color='orange', alpha=0.2)
```

Parameters:

- **fig** (go.Figure, required) — 대상 Figure
- **center** (tuple/list, required) — 중심 (x, y, z)
- **half_extents** (tuple/list, required) — 각 로컬 축 방향 절반 크기 (h_x, h_y, h_z)
- **rotation** (np.ndarray, required) — 3×3 회전 행렬 (열 = 로컬 축)
- **color** (str, default: 'orange') — 면 색상
- **alpha** (float, default: 0.2) — 투명도

Example:

```
angle = np.radians(25)
R = np.array([[np.cos(angle), -np.sin(angle), 0],
              [np.sin(angle),  np.cos(angle), 0],
              [0,             0,             1]])
draw_obb(fig, center=(0, 0, 0),
          half_extents=(2, 1, 1.5), rotation=R)
```

10 Quick Reference

Category	Function	Description
Figure	figure2d	2D Figure 생성 (orthographic)
	figure3d	3D Figure 생성 (cube aspect)
	setCam	카메라 설정 (LookAt)
Vector	draw_vec2d	2D 벡터 화살표
	draw_vec3d	3D 벡터 화살표
Point	draw_points	점 리스트 (2D/3D)
	draw_points_in_matrix	행렬 열을 점으로 표시
Matrix	draw_mat22	2×2 행렬 평행사변형
	draw_mat33	3×3 행렬 평행육면체
	draw_space_mat22	2×2 변환 격자 공간
Shape	draw_polygons	다각형 리스트
	draw_circle_2d	2D 원
	draw_circle_3d	3D 공간 원
Line	draw_line	선분 / 반직선 / 직선
Plane	draw_plane	세 점 → 삼각형 + 원
	draw_plane_from_normal	점 + 법선 → 원 + 법선 화살표
Bounding	draw_bounding_sphere	바운딩 구
	draw_aabb	AABB
	draw_obb	OBB